Chap. 1 : Le noyau Linux

Une des caractéristiques les plus intéressantes du noyau Linux est qu'il n'est pas un produit commercial, il est issu d'un projet collaboratif développé sur Internet. Bien que Linus Torvalds, qui a développé la première version du noyau Linux, reste le premier responsable de sa maintenance.

Le noyau Linux est diffusé sous licence GNU General Public License (GPL) version 2.0.

1. Les versions du noyau Linux

Le premier noyau Linux a été créé en 1991 par Linus Torvalds. Il a évolué au cours de temps et évolue encore.

Jusqu'à la version 2.6, les différentes versions du noyau Linux sont désignées par un numéro composé de trois séquences de chiffres délimitées par un point :

- la première séquence est le numéro de la version majeure. Ce numéro a été modifié uniquement pour des changements majeurs dans le code et le concept du noyau, en 1994 (version 1.0) et en 1996 (version 2.0) ;
- la deuxième séquence est le numéro de la version mineure. Un nombre pair indique qu'il s'agit d'un noyau stable, un nombre impair qu'il s'agit d'un noyau en développement.
- la troisième séquence est le numéro de révision. Ce numéro est incrémenté chaque fois qu'une nouvelle version du noyau est diffusée.

Depuis la publication de la version 2.6.0 :

- Les deux premiers numéros, « 2.6 », sont restés inchangés depuis décembre 2003 ;
- Le troisième numéro est la version courante du noyau ;
- Les développeurs du noyau ont ensuite introduit une quatrième séquence, le numéro de la version stable. Sa première utilisation date de la version du noyau 2.6.8

Le 29 mai 2011, Linus Torvalds annonça la version 3.0 du noyau, en l'honneur du 20e anniversaire de Linux. Cette version 3.0 n'est qu'une simple évolution de la 2.6.39.

2. Modification et configuration du noyau

Le noyau Linux est diffusé avec une configuration générique conçue pour supporter n'importe quelle application sur n'importe quel matériel.

Cette configuration générique comprend de nombreux pilotes de périphériques, mais aussi des paramètres pour le noyau. On peut modifier ces paramètres afin d'adapter le noyau à des besoins spécifiques, augmenter les performances, renforcer la sécurité, ou encore la fiabilité du système.

Dans certains cas, la modification du noyau est nécessaire afin d'ajouter de nouveaux pilotes de périphériques. Le code source du pilote ajouté doit être intégré dans les structures de données du noyau. Ceci peut exiger la re-compilation du noyau.

Il existe quatre méthodes pour intervenir sur la configuration d'un noyau Linux :

- Modification dynamique des paramètres de configuration du noyau ;
- construction d'un noyau à partir de zéro (compilation du code source, avec éventuellement des modifications et des ajouts) ;
- Chargement de modules dans un noyau existant, à la volée ;
- Passage de paramètres en utilisant le chargeur de démarrage : LILO ou GRUB.

Ces méthodes sont applicables dans des situations différentes. La modification dynamique des paramètres est la plus facile et la plus courante, tandis que la

construction d'un noyau à partir des fichiers sources est la plus difficile et la moins souvent nécessaire.

3. La modification dynamique des paramètres du noyau

Le noyau peut être ajusté dynamiquement à travers des paramètres du système. Ces paramètres sont accessibles et modifiables à partir des fichiers du répertoire **/proc/sys.**

Exemple :

Pour modifier le nombre maximal de fichiers que le système peut ouvrir simultanément, on peut modifier le fichier **/proc/sys/fs/file-max** de la façon suivante :

```
# echo 32768 > /proc/sys/fs/file-max
```

Tableau 1. Quelques paramètres ajustables du noyau Linux

Fichier	Description
/proc/sys/fs/file-max	Indique le nombre maximal de fichiers que le noyau peut manipuler simultanément
/proc/sys/kernel/ctrl-alt-del	Contrôle la gestion de la séquence Ctrl-Alt-Supp du clavier. S'il contient la valeur zéro, Ctrl-Alt-Supp est capturé et envoyé au programme init pour relancer le système correctement.
/proc/sys/net/ipv4/icmp_echo_ignore_all	Bloque les réponses au ping.
/proc/sys/net/ipv4/icmp_echo_ignore_broadcasts	Ignore les messages de diffusion ICMP (Internet Control Message Protocol)
/proc/sys/net/ipv4/ip_forward	Active ou désactive le routage (forwarding) entre les cartes réseaux. Activer le routage est nécessaire pour faire fonctionner le système comme un routeur.
/proc/sys/kernel/hostname	Permet de changer le nom de la machine.

Puisque le système de fichiers **/proc** est virtuel et n'a d'existence qu'au niveau de la mémoire centrale, la modification des fichiers qu'il contient est temporaire, et sera perdue lors du redémarrage du système.

Pour mémoriser les paramètres à appliquer au démarrage du système, on utilise le fichier **/etc/sysctl.conf**.

Si le fichier **/etc/sysctl.conf** contient la ligne :

```
net.ipv4.ip_forward = 0
```

Alors au démarrage du système le paramètre **ip_forward** aura pour valeur 0, donc le routage ne sera pas activé.

En cours d'exécution, les paramètres du noyau peuvent aussi être modifiés par la commande `sysctl` :

```
# sysctl –w net.ipv4.ip_forward=1
```

Cette commande donne au paramètre **`ip_forward`** la valeur 1, ce qui a pour effet d'activer le routage.

4. Construction d'un noyau Linux

On peut être amené à compiler et installer un nouveau noyau Linux pour intégrer des correctifs (patchs), des pilotes de périphériques ou de nouvelles fonctionnalités. Le code source et les correctifs des noyaux Linux sont disponibles sur le site www.kernel.org.

Exemple :

Dans l'exemple ci-dessous, le code source du noyau 2.6.35 a été téléchargé, à partir du lien www.kernel.org/pub/linux/kernel/v2.6, sous forme d'une archive nommée `tar.bz2`. Le répertoire `linux-2.6.35.5/` est créé à l'issue de la décompression et de l'extraction de cet archive par la commande `tar xjfv linux-2.6.35.5.tar.bz2`. À l'intérieur de ce répertoire se trouve l'arborescence du code source du noyau Linux 2.6.35.

```
$ tar xjfv linux-2.6.35.5.tar.bz2
$ cd linux-2.6.35.5/
$ ls
arch COPYING crypto drivers fs init Kbuild lib Makefile net REPORTING-BUGS
scripts sound usr block CREDITS Documentation firmware include ipc kernel
MAINTAINERS mm README samples security tools virt
```

4.1 Configuration du noyau

Les informations sur la configuration du noyau sont stockées dans le fichier `.config` situé à la racine du répertoire source du noyau.

Il est déconseillé de modifier ce fichier manuellement. Linux offre plusieurs outils permettant de configurer le noyau et d'écrire le résultat dans le fichier `.config`.

4.1.1 Make config

L'outil le plus élémentaire est `make config`. Il s'exécute en mode console. L'utilisateur doit ensuite spécifier toutes les options de configuration. `make config` demande pour chaque fonction si elle doit ou non être activée, en proposant quatre choix possibles sous la forme [Y / m / n /?] :

- `Y` (« yes ») pour intégrer la fonction directement dans le noyau. C'est le choix par défaut, il peut être sélectionné en appuyant simplement sur la touche **[Entrée]** ;
- `m` (« module ») pour construire un module qui va être chargé de façon dynamique ;
- `n` (« no ») pour ne pas activer la fonction ;
- `?` pour afficher un message décrivant la fonction.

4.1.2 Make oldconfig

Le noyau contient près de deux mille options de configuration, et répondre à toutes les questions prend beaucoup de temps. Heureusement, il y a un moyen plus rapide de configurer le noyau : utiliser l'outil `make oldconfig` qui construit une configuration basée sur une configuration pré-construite.

Exemple.

On va construire une configuration pour le nouveau noyau 2.6.35.5 à partir de la configuration pré-construite pour le noyau courant (version 2.6.32.24) :

```
$ make oldconfig
scripts/kconfig/conf -o arch/x86/Kconfig
#
# using defaults found in /boot/config-2.6.32-24-generic
#
/boot/config-2.6.32-24-generic:556:warning: symbol value 'm' invalid for
```

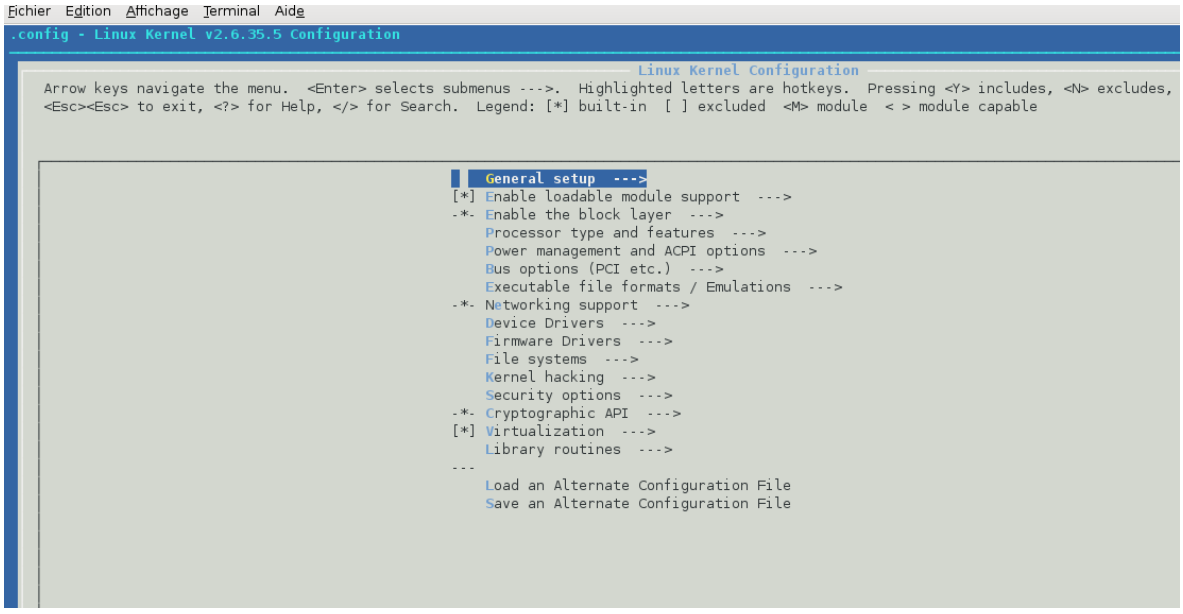
```
PCCARD_NONSTATIC
/boot/config-2.6.32-24-generic:3031:warning: symbol value 'm' invalid for
MFD_WM831X
/boot/config-2.6.32-24-generic:3032:warning: symbol value 'm' invalid for
MFD_WM8350
/boot/config-2.6.32-24-generic:3033:warning: symbol value 'm' invalid for
MFD_WM8350_I2C
/boot/config-2.6.32-24-generic:3038:warning: symbol value 'm' invalid for
AB3100_CORE
/boot/config-2.6.32-24-generic:3505:warning: symbol value 'm' invalid for
FB_VESA
/boot/config-2.6.32-24-generic:4174:warning: symbol value 'm' invalid for
MMC_RICOH_MMC
*
* Restart config...
*
*
* General setup
*
Prompt for development and/or incomplete code/drivers (EXPERIMENTAL)
[Y/n/?] y
Cross-compiler tool prefix (CROSS_COMPILE) [] (NEW)
```

4.1.3 Make menuconfig

Il existe trois autres outils interactifs pour la configuration du noyau.

Le premier outil est `make menuconfig`, il permet une configuration en mode console, la navigation entre les options de configuration est faite à l'aide des touches fléchées du clavier (*figure 1*).

Figure 1. Outil `make menuconfig`



4.1.4 Make gconfig, make xconfig

Les deux autres outils, make gconfig et make xconfig, utilisent des interfaces graphiques permettant de modifier la configuration du noyau.

Pour ces deux outils on utilise la souris pour naviguer dans les menus et sélectionner des options

4.2.Compilation du noyau

Une fois que la configuration du noyau est créée, on peut compiler le noyau en utilisant la commande make :

```
$ make
```

Cette commande a pour effet de compiler le noyau avec la configuration définie dans l'étape précédente, ainsi que tous les modules nécessaires à cette configuration.

Si la compilation du noyau se termine sans erreur, le résultat est un fichier binaire – le fichier image du noyau – qui doit être installé avant qu'on puisse l'utiliser au démarrage de la machine.

4.3. Installation et démarrage du nouveau noyau

L'installation du nouveau noyau peut être réalisée soit en utilisant des scripts offerts par la distribution installée, soit de façon manuelle.

4.3.1. Utilisation des scripts d'installation d'une distribution

La plupart des distributions Linux sont livrées avec un script appelé `installkernel` qui peut être utilisé lors de l'installation d'un nouveau noyau compilé. Ce script permet de copier le nouveau noyau dans le répertoire approprié et de modifier le chargeur de démarrage afin que ce nouveau noyau puisse être sélectionné lors du démarrage du système.

Si on a choisi d'utiliser des modules externes dynamiques, on doit d'abord les installer avec la commande:

```
# make modules_install
```

Ceci permet de placer tous les modules compilés dans le répertoire approprié dans l'arborescence standard Linux, où le noyau pourra les retrouver en cas de besoin. Dans notre exemple les modules dynamiques sont placés dans le répertoire `/lib/modules/2.6.35.5`.

Une fois les modules installés correctement, l'image du nouveau noyau doit être aussi installée par la commande :

```
# make install
```

Cette commande déclenche le processus suivant :

1. Vérification de la bonne construction du nouveau noyau.
2. Installation de la partie statique du noyau dans le répertoire /boot

3. Création de toutes les images RAM Disk nécessaires, en utilisant les modules installés lors de l'exécution de la commande `make modules_install`.
4. Ajout d'une entrée dans le fichier de configuration du chargeur de démarrage pour le nouveau noyau.

Une fois ce processus terminé, le noyau est installé avec succès, on peut donc redémarrer et tester la nouvelle image du noyau.

4.3.2. Installation manuelle

Si la distribution ne dispose pas du script `installkernel`, ou si l'on souhaite tout simplement faire l'installation manuellement, on doit procéder de la manière suivante :

Installer les modules :

```
# make modules_install
```

Le noyau doit être copié dans le répertoire `/boot`. Pour un noyau basé sur l'architecture 386, on exécute la commande :

```
# cp arch/i386/boot/bzImage /boot/vmlinuz-2.6.35.5
```

```
# cp System.map /boot/System.map-2.6.35.5
```

On doit ensuite modifier le fichier de configuration du chargeur de démarrage afin qu'il puisse reconnaître le nouveau noyau.

Si le processus de démarrage ne fonctionne pas correctement, c'est généralement parce qu'une image `initrd` est nécessaire.

Une image `initrd` est généralement construite par la commande `mkinitrd`.

```
mkinitrd -o /boot/initrd-2.6.35.5.img /lib/modules/2.6.35.5
```

5. Mise à jour d'un noyau

Il est facile de mettre à jour un noyau d'une version ancienne tout en conservant toutes les options de la configuration précédente.

Tout d'abord, dans le répertoire source du noyau on sauvegarde le fichier `.config`.

```
$ cd linux-2.6.35.5
```

```
$ cp .config ../good_config
```

Cinq étapes sont nécessaires pour mettre à niveau un noyau déjà compilé et installé :

1. Obtenir le nouveau code source.
2. Appliquer les modifications sur l'ancienne arborescence du code source du noyau pour le mettre à niveau.
3. Reconfigurer le nouveau noyau en se basant sur la configuration précédente.
4. Compiler le nouveau noyau.
5. Installer le nouveau noyau.

Les trois dernières étapes sont identiques à celles décrites précédemment, seules les deux premières sont donc présentées ici.

Exemple :

La version 2.6.35.5 du noyau de Linux est déjà compilée et installée. Il s'agit maintenant de mettre à jour ce noyau vers la version 2.6.35.7.

On a donc besoin de télécharger deux patchs : un patch pour passer de la version 2.6.35.5 à la version 2.6.35.6, puis un autre patch pour passer de la version 2.6.35.6 à la version 2.6.35.7.

On les trouve à la page `www.kernel.org/pub/linux/kernel/v2.6/incr/`.

Comme ils sont compressés, la première chose à faire est de les décompresser avec la commande bzip2 :

```
$ bzip2 -dv patch-2.6.35.5-6.bz2
$ bzip2 -dv patch-2.6.35.6-7.bz2
```

Maintenant, il faut appliquer les fichiers de patch dans le répertoire du noyau. On se place dans le répertoire `linux-2.6.35.5` et on lance la commande `patch` :

```
$ cd linux-2.6.35.5
$ patch -p1 < ../patch-2.6.35.5-6
```

On peut examiner le fichier `Makefile` pour voir le changement de version du noyau :

```
$ head Makefile
```

On utilise à nouveau la commande `patch` pour passer de la version 2.6.35.6 à la version 2.6.35.7 :

```
$ patch -p1 < ../patch-2.6.35.6-7
```

6. Modules du noyau

Le noyau Linux supporte un chargement dynamique des modules : il est possible de les charger ou de les supprimer quand le noyau est en cours d'exécution.

Les modules du noyau sont stockés dans le répertoire `/lib/modules/version_noyau`, où version_noyau est la version du noyau Linux en cours (retournée par la commande `uname -r`).

6.1 lsmod

La commande `lsmod` liste les modules actuellement chargés, avec leurs dépendances éventuelles.

```
# lsmod
Module Size Used by
fglrx 1482380 70
nls_iso8859_1 8192 1
nls_cp437 9856 1
vfat 16128 1
fat 52636 1 vfat
…
```

Dans l'exemple précédent le module `fat` est utilisé par le module `vfat`.

`lsmod` ne fait que remettre en forme le contenu du fichier virtuel `/proc/modules`.

```
# cat /proc/modules
fglrx 1482380 60 - Live 0xf9a55000 (P)
nls_iso8859_1 8192 0 - Live 0xf94d0000
nls_cp437 9856 0 - Live 0xf94ed000
…
```

6.2 depmod

La commande `depmod` met à jour l'arbre des dépendances entre les modules en modifiant le fichier `modules.dep`.

```
# grep vfat /lib/modules/$(uname -r)/modules.dep
```

```
/lib/modules/2.6.22.17-0.1-default/kernel/fs/vfat/vfat.ko:
/lib/modules/2.6.22.17-0.1-default/kernel/fs/fat/fat.ko
```

Le module `vfat.ko` dépend du module `fat.ko`. Il faut donc que le module `fat.ko` soit chargé en premier pour que `vfat.ko` fonctionne.

6.3 modinfo

La commande `modinfo` fournit toutes les informations nécessaires sur un module :

- le nom du fichier correspondant,
- une description du module,
- son auteur,
- sa licence,
- ses dépendances,
- ses paramètres,
- ses alias matériels.

```
# modinfo vfat
filename: /lib/modules/2.6.22.17-0.1-default/kernel/fs/fat/fat.ko
license: GPL
srcversion: 886B1B65F96E53415B5811C
depends:
supported: yes
vermagic: 2.6.22.17-0.1-default SMP mod_unload 586
```

6.4 insmod

La commande `insmod` charge un module donné sans gérer les dépendances. Elle prend en paramètre un nom de module, avec son éventuel chemin.

```
# insmod vfat.ko
insmod: error inserting 'vfat.ko': -1 Unknown symbol in module
```

Une erreur se produit car ce module dépend de la présence de `fat.ko`.

Dans ce second exemple, le module `fat` est tout d'abord chargé, puis le module `vfat`. Il n'y a pas d'erreur car toutes les dépendances sont présentes.

```
# insmod fat.ko
# insmod vfat.ko
# lsmod|grep fat
vfat 16128 0
fat 52636 1 vfat
```

6.5 rmmod

La commande `rmmod` décharge le module donné. C'est l'inverse de `insmod` et, comme cette dernière, `rmmod` ne gère pas les dépendances :

```
# rmmod fat
ERROR: Module fat is in use by vfat
```

Dans ce dernier exemple, le module `vfat` est déchargé, puis le module `fat`, dans cet ordre.

```
# rmmod vfat
# rmmod fat
```

6.6 modprobe

La commande modprobe charge le module donné ainsi que toutes ses dépendances et des paramètres contenus dans `/etc/modprobe.conf`. Le paramètre `-r` permet de décharger un modules et ceux qui en dépendent (s'ils ne sont pas utilisés).

Le chargement du module `vfat` à l'aide de `modprobe` va automatiquement charger le module `fat`.

```
# lsmod|grep fat
# modprobe vfat
# lsmod|grep fat
vfat 16128 0
fat 52636 1 vfat
```

vous pouvez tenter de décharger `vfat` et les modules dont il dépend s'ils ne sont plus utilisés.

```
# modprobe -r vfat
# lsmod|grep fat
```

7. Passage de paramètres au noyau à partir du chargeur de démarrage

On peut avoir besoin de passer quelques options au noyau Linux lors du démarrage du système. Le chargeur de démarrage (LILO ou GRUB) est responsable du transfert de ces options au noyau.

Pour que ces options soient prises en compte à chaque démarrage du système, on doit les ajouter dans les fichiers de configurations /etc/lilo.conf ou /boot/grub/grub.conf, selon le chargeur de démarrage utilisé.

On peut aussi utiliser l'invite de commandes du chargeur de démarrage

EXEMPLE

Pour indiquer à LILO de charger le noyau spécifié par l'étiquette « linux », d'utiliser la partition racine /dev/sda1 et de détecter deux cartes Ethernet, on utilise la commande LILO suivante:

```
LILO: linux root=/dev/sda1 ether=0,0,eth0 ether=0,0,eth1
```

Pour indiquer la même chose à GRUB on utilise la commande :

```
grub> kernel /vmlinuz root=/dev/sda1 ether=0,0,eth0 ether=0,0,eth1
grub> boot
```